

“Plain” object-oriented vs. ParaPy code – cheat sheet

Changes in Code Structure

OOP code

```
import numpy as np
import math

def estimate_cD0(A_ref: float, b: float, phi_LE: float) → float:
    c_ref = A_ref / b # mean chord length
    Re = 1e+6 * c_ref # representative Reynolds number
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated!
    return c_D0

class Wing:
    def __init__(self, A_ref, b, phi_LE=0, alpha0=0, e=0.85):
        self.A_ref = A_ref
        self.b = b
        self.phi_LE = phi_LE
        self.alpha0 = alpha0
        self.e = e

        self.AR = b**2 / A_ref
        self.cD0 = estimate_cD0(A_ref, b, phi_LE)
        self.clalpha = 2 + 0 * (self.AR + self.phi_LE)

    def get_lift(self, alpha):
        return self.clalpha \
            * (math.radians(alpha) - math.radians(self.alpha0))

    def get_drag(self, cL):
        return self.cD0 + cL**2 / (np.pi * self.AR * self.e)

    def get_polar(self, alphas) → np.array:
        results = []
        for alpha in alphas:
            cL = self.get_lift(alpha)
            cD = self.get_drag(cL)
            results.append([cL, cD])

        return np.array(results)

testwing = Wing(A_ref=30, b=20, phi_LE=15, alpha0=-0.6, e=0.85)

alpha = 1.5
alphas_polar = np.linspace(0, 3, num=16)

cL = testwing.get_wing_lift(alpha)
cD = testwing.get_wing_drag(cL)
polar = testwing.get_polar(alphas_polar)
```

unbound methods work in the same way
(only useful if they're “external” code or
are also needed outside of the class)

No `__init__()`
Input arguments are declared as class
attributes (See also next page!)

Internally-computed variables get their own
`@Attribute` methods, so they can be traced
and computed separately, as needed

„plain“ instance methods work exactly as they
used to. Unless they require extra input or you
need to trigger them explicitly, it's usually smarter
to use `@Attributes`, though!

creating an instance and calling its
methods works exactly the same

ParaPy code

```
import numpy as np
import math
from parapy import core as ppc

def estimate_cD0(A_ref: float, b: float, phi_LE: float) → float:
    c_ref = A_ref / b # mean chord length
    Re = 1e+6 * c_ref # representative Reynolds number
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated estimate!
    return c_D0

class Wing_PP(ppc.Base):
    A_ref = ppc.Input()
    b = ppc.Input()
    phi_LE = ppc.Input(0)
    alpha0 = ppc.Input(0)
    e = ppc.Input(0.85)

    @ppc.Attribute
    def AR(self):
        return self.b**2 / self.A_ref

    @ppc.Attribute
    def cD0(self):
        return estimate_cD0(self.A_ref, self.b, self.phi_LE)

    @ppc.Attribute
    def clalpha(self):
        return 2 + 0 * (self.AR + self.phi_LE)

    def get_lift(self, alpha):
        return self.clalpha \
            * (math.radians(alpha) - math.radians(self.alpha0))

    def get_drag(self, cL):
        return self.cD0 + cL**2 / (np.pi * self.AR * self.e)

    def get_polar(self, alphas) → np.array:
        results = []
        for alpha in alphas:
            cL = self.get_lift(alpha)
            cD = self.get_drag(cL)
            results.append([cL, cD])
        return np.array(results)

testwing_pp = Wing_PP(A_ref=30, b=20, phi_LE=15, alpha0=-0.5, e=0.85)

alpha = 1.5
alphas_polar = np.linspace(0, 3, num=16)

cL = testwing_pp.get_wing_lift(alpha)
cD = testwing_pp.get_wing_drag(cL)
polar = testwing_pp.get_polar(alphas_polar)
```

“Plain” object-oriented vs. ParaPy code – cheat sheet

Differences in Data Handling

OOP code

```
import numpy as np
import math

def estimate_cD0(A_ref: float, b: float, phi_LE: float) → float:
    c_ref = A_ref / b # mean chord length
    Re = 1e+6 * c_ref # representative Reynolds number
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated!
    return c_D0
```

```
class Wing:
    def __init__(self, A_ref, b, phi_LE=0, alpha0=0, e=0.85):
        self.A_ref = A_ref
        self.b = b
        self.phi_LE = phi_LE
        self.alpha0 = alpha0
        self.e = e

        self.AR = b**2 / A_ref
        self.cD0 = estimate_cD0(A_ref, b, phi_LE)
        self.clalpha = 2 + 0 * (self.AR + self.phi_LE)

    def get_lift(self, alpha):
        return self.clalpha \
            * (math.radians(alpha) - math.radians(self.alpha0))

    def get_drag(self, cL):
        return self.cD0 + cL**2 / (np.pi * self.AR * self.e)

    def get_polar(self, alphas) → np.array:
        results = []
        for alpha in alphas:
            cL = self.get_lift(alpha)
            cD = self.get_drag(cL)
            results.append([cL, cD])

        return np.array(results)
```

```
testwing = Wing(A_ref=30, b=20, phi_LE=15, alpha0=-0.6, e=0.85)
```

```
alpha = 1.5
alphas_polar = np.linspace(0, 3, num=16)
```

```
cL = testwing.get_lift(alpha)
cD = testwing.get_drag(cL)
polar = testwing.get_polar(alphas_polar)
```

```
testwing.A_ref = 10 # This is a terrible idea!
# ...unless you write some extra code to make sure that
# AR, cD0 and clalpha are updated afterwards...
```

`__init__()` is called implicitly at instantiation and can calculate and store whatever attributes are necessary for the object to work, as `self.<attr>`

`__init__()` computes intermediate variables once and stores results

All methods defined within the class are available as instance attributes

If you change an input attribute after the fact, that can create an inconsistent state!

The ParaPy base class adds its own `__init__()` which process all inputs, so you should not write one!

Code to compute non-input attributes is declared in separate methods. ParaPy finds their dependencies and (re-)computes if (and only if) needed. You never need to call these methods!

The same, except: ParaPy **@Attributes** can't be called directly – you can only access their value, and ParaPy ensures it's updated

ParaPy tracks when inputs change and updates the internal state as needed

ParaPy code

```
import numpy as np
import math
from parapy import core as ppc

def estimate_cD0(A_ref: float, b: float, phi_LE: float) → float:
    c_ref = A_ref / b # mean chord length
    Re = 1e+6 * c_ref # representative Reynolds number
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated estimate!
    return c_D0
```

```
class Wing_PP(ppc.Base):
    A_ref = ppc.Input()
    b = ppc.Input()
    phi_LE = ppc.Input(0)
    alpha0 = ppc.Input(0)
    e = ppc.Input(0.85)

    @ppc.Attribute
    def AR(self):
        return self.b**2 / self.A_ref

    @ppc.Attribute
    def cD0(self):
        return estimate_cD0(self.A_ref, self.b, self.phi_LE)

    @ppc.Attribute
    def clalpha(self):
        return 2 + 0 * (self.AR + self.phi_LE)

    def get_lift(self, alpha):
        return self.clalpha \
            * (math.radians(alpha) - math.radians(self.alpha0))

    def get_drag(self, cL):
        return self.cD0 + cL**2 / (np.pi * self.AR * self.e)

    def get_polar(self, alphas) → np.array:
        results = []
        for alpha in alphas:
            cL = self.get_lift(alpha)
            cD = self.get_drag(cL)
            results.append([cL, cD])
        return np.array(results)
```

```
testwing_pp = Wing_PP(A_ref=30, b=20, phi_LE=15, alpha0=-0.5, e=0.85)
```

```
alpha = 1.5
alphas_polar = np.linspace(0, 3, num=16)
```

```
cL = testwing_pp.get_lift(alpha)
cD = testwing_pp.get_drag(cL)
polar = testwing_pp.get_polar(alphas_polar)
```

```
testwing_pp.A_ref = 10
cD_new = testwing_pp.get_drag(cL)
```

@ppc.Attribute definitions look like methods but can **do nothing** but compute one variable, and we never call them – so they're **named after the variable they compute**

Object-Oriented code structure – cheat sheet

Information flow

Outer scope

```
import numpy as np
```

```
[...]
```

```
testwing = Wing(A_ref=30, b=20, phi_LE=15, alpha0=-0.6, e=0.85)
```

```
alpha = 1.5  
alphas_polar = np.linspace(0, 3, num=16)
```

```
cL = testwing.get_lift(alpha)  
cD = testwing.get_drag(cL)
```

```
polar = testwing.get_polar(alphas_polar)
```

```
print(testwing.clalpha)  
# object-internal data remains available!
```

The outer scope provides the input data, once, and the object “knows” what to do with it.

The object can be passed as an argument to other methods/classes, or even saved to disk and loaded later.

If we want to change inputs, it's safer to create a new object:

```
testwing.A_ref = 10 # This is a terrible idea!  
# ...unless you write some extra code to make sure that  
# AR, cD0 and clalpha are updated afterwards...  
  
# better:  
testwing2 = Wing(A_ref=10, b=20, phi_LE=15, alpha0=-0.6, e=0.85)  
# Now we can keep both wings around, and each wing contains its own data
```

The object returned includes all methods and other attributes defined in the class, including those defined by `__init__()`

Classes are **instantiated**, not called!
This produces an *object* which is an *instance* of the class

Object-specific data

Instance methods can be *called*, just like all other methods

Inner scope

```
import numpy as np  
import math
```

```
def estimate_cD0(A_ref: float, b: float, phi_LE: float) -> float:  
    c_ref = A_ref / b # mean chord length  
    Re = 1e+6 * c_ref # representative Reynolds number  
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated estimate!  
    return c_D0
```

```
class Wing:
```

```
def __init__(self, A_ref, b, phi_LE=0, alpha0=0, e=0.85):  
    self.A_ref = A_ref  
    self.b = b  
    self.phi_LE = phi_LE  
    self.alpha0 = alpha0  
    self.e = e
```

```
self.AR = b**2 / A_ref  
self.cD0 = estimate_cD0(A_ref, b, phi_LE)  
self.clalpha = 2 + 0 * (self.AR + self.phi_LE)
```

```
def get_lift(self, alpha):  
    return self.clalpha * (math.radians(alpha) -  
                           math.radians(self.alpha0))
```

```
def get_drag(self, cL):  
    return self.cD0 + cL**2 / (np.pi * self.AR * self.e)
```

```
def get_polar(self, alphas) -> np.array:  
    results = []  
    for alpha in alphas:  
        cL = self.get_lift(alpha)  
        cD = self.get_drag(cL)  
        results.append([cL, cD])  
  
    return np.array(results)
```

<builtin `__new__()`>
creates a new object
adds all methods from the class
calls its `__init__()` method
returns the initialized object

All methods **inside** the class can access all *attributes* and *methods* as `self.<name>`

Direct inputs *and* intermediate results are stored and are available to all internal methods

ParaPy code structure – cheat sheet

Information flow

Outer scope

User-class scope

ParaPy-internal

```
import numpy as np
import math
```

```
testwing_pp = Wing_PP(A_ref=30, b=20, phi_LE=15,
                      alpha0=-0.5, e=0.85)
```

```
alpha = 1.5
alphas_polar = np.linspace(0, 3, num=16)
```

```
cL = testwing_pp.get_lift(alpha)
cD = testwing_pp.get_drag(cL)
polar = testwing_pp.get_polar(alphas_polar)
```

```
print(testwing_pp.clalpha)
# object-internal data remains available!
```

The outer scope provides the input data, once, and the object “knows” what to do with it.

... and we can assign new values to the inputs, without worrying about consistency!

Modified Input

```
testwing_pp.A_ref = 10
cD_new = testwing_pp.get_drag(cL)
```

As with OOP code, the object includes all methods, inputs and other attributes defined in the class (plus special ParaPy additions)

ParaPy classes are not too different from „normal“ classes

```
import math
from parapy import core as ppc

def estimate_cD0(A_ref: float, b: float, phi_LE: float) → float:
    c_ref = A_ref / b # mean chord length
    Re = 1e+6 * c_ref # representative Reynolds number
    c_D0 = 0.05 / (Re/1e+6)**(1/5) # accurately calibrated estimate!
    return c_D0
```

```
class Wing_PP(ppc.Base):
```

```
    A_ref = ppc.Input()
    b = ppc.Input()
    phi_LE = ppc.Input(0)
    alpha0 = ppc.Input(0)
    e = ppc.Input(0.85)
```

store input values or use default

```
@ppc.Attribute
def AR(self):
    return self.b**2 / self.A_ref
```

```
@ppc.Attribute
def cD0(self):
    return estimate_cD0(self.A_ref, self.b, self.phi_LE)
```

```
@ppc.Attribute
def clalpha(self):
    return 2 + 0 * (self.AR + self.phi_LE)
```

```
def get_lift(self, alpha):
    return self.clalpha \
        * (math.radians(alpha) - math.radians(self.alpha0))
```

```
def get_drag(self, cL):
    return self.cD0 + cL**2 / (np.pi * self.AR * self.e)
```

```
def get_polar(self, alphas) → np.array:
    results = []
    for alpha in alphas:
        cL = self.get_lift(alpha)
        cD = self.get_drag(cL)
        results.append([cL, cD])
    return np.array(results)
```

```
parapy.core.Base
# ...provides special __init__() method,
# registers Inputs, @Attributes, @Parts...
```

```
class Attribute(AbstractSlot):
    # can be used just like a plain attribute in
    # OOP, with a plain value (e.g. 'self.AR')
    #
    # ...but actually:
    # - stores the provided calculation method
    # - computes value only(!) if needed, by
    #   calling the calculation method
    # - Stores the result for next time
    # - Keeps a list of dependencies, to decide
    #   when the stored value is outdated
```

ParaPy registers all changes to Inputs and Attributes. Next time any dependent attributes (e.g. AR or cD0) are accessed, they are recomputed

The object is „augmented“ by ParaPy such that @Inputs and @Attributes are tracked and evaluated when needed, and only then.

The class definition is mostly a declaration for ParaPy to construct a dependency graph and keep the object consistent when inputs change.